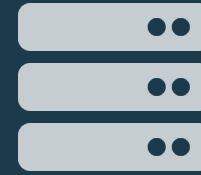


FUNDAMENTOS DE INFRAESTRUTURA

Por que os computadores são compartilhados?

Da máquina dedicada às máquinas virtuais, até o Docker — e o Linux, o SSH e os comandos que sustentam tudo isso por trás dos bastidores.



O que vamos ver hoje



1. Por que compartilhar computadores?

O custo de um servidor por aplicação



3. Linux nos servidores

Por que ele domina a internet — e por que não tem tela



5. SSH

A ponte até um servidor do outro lado do mundo



7. Grupos Linux

O segredo por trás do sudo



2. Uma linha do tempo

Máquina única → máquinas virtuais → containers



4. Comandos básicos do Linux

cd, ls, cat, nano, mkdir, rm



6. Docker por dentro

Dockerfile, imagem, container, Docker Hub

1

Por que compartilhar computadores?

O custo real de dar um servidor inteiro para cada aplicação



O problema de “1 aplicação = 1 servidor”



Custo alto

Comprar, energizar e manter um servidor físico para cada aplicação sai caro — hardware, eletricidade, espaço no data center.



Maior parte do tempo, ocioso

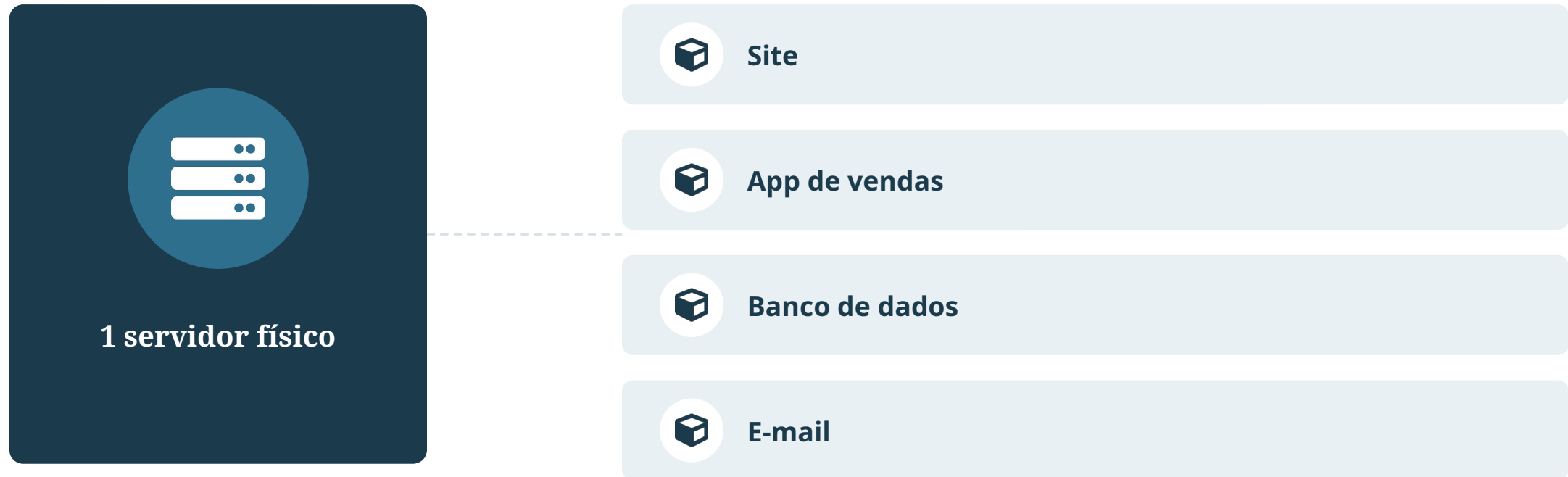
A maioria das aplicações não usa 100% da CPU e da memória o tempo todo. O servidor fica “parado” na maior parte do dia.



Demora para crescer

Precisa de uma aplicação nova? Comprar, entregar e configurar uma máquina física inteira leva dias ou semanas.

A ideia central: dividir o mesmo hardware



Cada aplicação usa só a fatia de CPU e memória que precisa, **aproveitando a capacidade que antes ficava ociosa.**

2

Uma linha do tempo do compartilhamento

De um extremo (isolamento total) a outro (compartilhamento inteligente)



Três marcos, uma mesma busca



Marco 1 — Uma aplicação por máquina

Cada aplicação recebe seu próprio servidor físico, só para ela.



Site

1 servidor dedicado



App

1 servidor dedicado



Banco de dados

1 servidor dedicado

✓ **Isolamento total** — um problema em uma máquina não afeta as outras.

✗ **Caro, lento para crescer e desperdiça capacidade ociosa (o problema do bloco 1).**

Marco 2 — Máquinas virtuais (VMs)

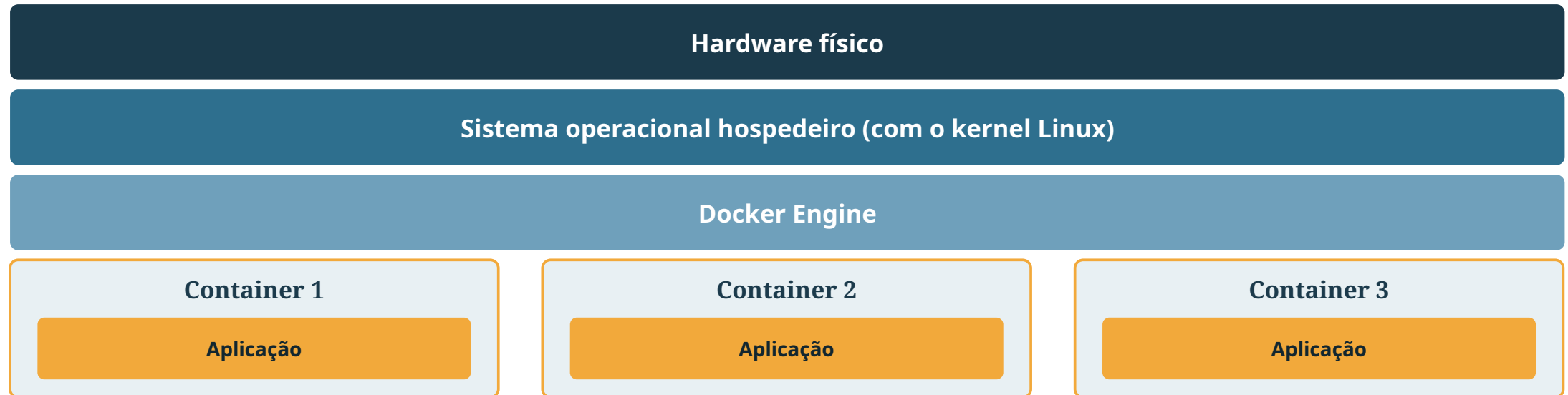
Um programa chamado hypervisor divide 1 servidor físico em várias máquinas virtuais — cada uma rodando seu próprio sistema operacional completo.



Cada VM carrega um sistema operacional inteiro — melhor uso do hardware, mas pesado e mais lento para iniciar.

Marco 3 — Containers (Docker)

Os containers compartilham o kernel do sistema operacional hospedeiro — cada um isola só a aplicação e suas dependências, sem carregar um SO inteiro.



Sem SO próprio por container: só a aplicação. Resultado: inicia em segundos, usa poucos recursos, e cabem muito mais containers no mesmo hardware.

Docker vs. Máquina Virtual: a diferença central

Máquina Virtual



Virtualiza o HARDWARE.
Cada VM carrega um sistema
operacional completo (guest OS).

Mais pesado • Inicia em minutos • Isolamento máximo

Docker (Container)



Virtualiza as bibliotecas e aplicações.
Os containers compartilham
o kernel do hospedeiro.

Muito mais leve • Inicia em segundos • Isolamento da aplicação

Uma analogia simples



VM = casas isoladas

Cada casa tem sua própria fundação, seu próprio encanamento, sua própria estrutura elétrica — construída do zero, independente das vizinhas.



Container = apartamentos

Todos os apartamentos compartilham a fundação e a estrutura do prédio (o kernel) — mas cada um tem sua porta trancada e seu próprio espaço isolado.

Recapitulando os três marcos

	1 app / máquina	Máquina virtual	Container (Docker)
Peso	Muito pesado	Pesado (SO completo)	Leve (só a app)
Velocidade de início	Minutos a horas	Minutos	Segundos
Isolamento	Total (física)	Alto (SO próprio)	Da aplicação
Custo de hardware	Muito alto	Médio	Baixo

3

Linux nos servidores da internet

Por que ele domina — e por que raramente tem tela



O sistema operacional dos servidores

A Maioria

dos sites do mundo rodam
em servidores Linux

Fonte: W3Techs, set/2026



Gratuito e de código aberto — sem custo de licença, mesmo em milhares de servidores.



Estável e seguro — servidores ficam meses ou anos ligados sem precisar reiniciar.



Leve — usa pouca memória e processamento — mais recursos sobram para a aplicação.



Comunidade gigante — suporte, documentação e ferramentas para qualquer necessidade.

Servidores raramente têm tela



*Sem monitor, mouse
nem interface gráfica*

A maioria dos servidores Linux roda em “modo texto”, sem nenhuma interface gráfica instalada.

Isso não é uma limitação — é intencional: cada MB de memória e cada ciclo de processamento economizado na interface gráfica é um recurso a mais disponível para a aplicação que realmente importa.

Por isso, comandos são fundamentais



Sem tela, não existe “clicar em um ícone”.

Toda tarefa — navegar entre pastas, ler um arquivo, criar um diretório, apagar algo — precisa ser feita digitando o **comando exato** em um terminal de texto.

É exatamente esse conjunto de comandos que vamos ver a seguir.

4

Comandos básicos do Linux

O vocabulário mínimo para sobreviver em um terminal



Navegar entre pastas



```
cd pasta
```

Entra em uma pasta. Use `cd ..` para voltar uma pasta acima, e `cd` sozinho para ir direto para a pasta do usuário.



```
ls
```

Lista os arquivos e pastas do diretório atual. Use `ls -la` para ver também os ocultos e os detalhes (dono, tamanho, data).

Ver e editar arquivos



```
cat arquivo
```

Mostra na tela o conteúdo de um arquivo de texto — não edita, só exhibe.



```
nano arquivo
```

Abre um editor de texto simples no terminal. Ctrl+O salva, Ctrl+X sai.

Criar e remover



```
mkdir pasta
```

Cria uma nova pasta (diretório) com o nome indicado.



```
rm arquivo
```

Remove um arquivo. Para remover uma pasta inteira, use `rm -r pasta`.



rm é definitivo: não existe “lixeira” no terminal. Um arquivo apagado com rm não volta.

RECAPITULANDO

os 6 comandos básicos

cd

Navega entre pastas

cat

Mostra o conteúdo de um arquivo

mkdir

Cria uma nova pasta

ls

Lista arquivos e pastas

nano

Edita um arquivo de texto

rm

Remove um arquivo (ou -r, uma pasta)

5

SSH: a ponte até o servidor remoto

Como acessar uma máquina que não está na sua frente



O que é o SSH?



SSH (Secure Shell) é um protocolo que conecta o terminal do seu computador ao terminal de outro computador, **através da internet, com tudo criptografado** — ninguém no meio do caminho consegue ler o que é digitado.

É como se o seu teclado e sua tela fossem “teleportados” para dentro do outro computador — tudo que você digita roda lá, não aqui.

SEM ELE, NÃO TEM COMO ACESSAR

Por que o SSH é essencial no dia a dia



Servidores ficam em data centers — muitas vezes em outro país — **sem teclado, sem mouse e sem monitor conectados**. O SSH é a única “porta de entrada”: é como todo programador acessa e administra esses servidores no dia a dia.

A sintaxe do comando

```
ssh usuario@endereco -p porta
```

usuario

a conta que você vai acessar no servidor remoto

endereco

o IP (ou nome) da máquina que você quer acessar

porta

cada servidor pode expor o SSH em uma porta diferente

6

Docker por dentro

Dockerfile → imagem → container → Docker Hub



Da receita ao prato pronto



Dockerfile — o código fonte



É um arquivo de texto com o passo a passo para montar o ambiente:

- qual sistema usar como base
- quais programas instalar
- quais arquivos copiar para dentro
- quais comandos rodar ao iniciar

Imagem — o código compilado



É o resultado de seguir a receita do Dockerfile: **um pacote “congelado” com tudo que a aplicação precisa para rodar** — sistema, bibliotecas, código, configurações.

A imagem em si não está “rodando” — é só o molde, pronto para ser usado quantas vezes for preciso.

Container — o programa rodando



É uma cópia da imagem **em execução** — o processo vivo da aplicação rodando, isolado dos demais.

Você pode rodar várias cópias (containers) da mesma imagem ao mesmo tempo, cada uma isolada das outras.

Docker Hub — a prateleira de imagens



É um repositório na nuvem cheio de imagens já prontas (*Nginx, MySQL, Ubuntu, Python...*) que qualquer pessoa pode baixar e usar, sem precisar montar tudo do zero.

```
docker pull nginx
```

Instalar e testar

1. Instalar o Docker

```
sudo apt update && sudo apt install -y docker.io
```

Em uma distribuição baseada em Debian/Ubuntu, instala o Docker pelo gerenciador de pacotes do sistema.

2. Testar a instalação

```
sudo docker run hello-world
```

Se essa imagem mínima de teste rodar e imprimir uma mensagem de sucesso, o Docker está instalado e funcionando.

docker ps — quem está rodando agora?

```
docker ps
```

Lista todos os containers em execução neste momento: nome, imagem usada, portas publicadas e há quanto tempo está rodando.

```
CONTAINER ID   IMAGE     COMMAND                  STATUS              PORTS              NAMES
a1b2c3d4e5f6   nginx    "nginx -g..."         Up 2 minutes       0.0.0.0:8080->80/tcp  meusite
```

7

Grupos Linux e o segredo do sudo

Por que às vezes docker ps funciona sozinho, e às vezes não



O que é um grupo no Linux?



No Linux, cada usuário pode pertencer a um ou mais grupos, e cada grupo carrega certas permissões.

Existe até um grupo chamado **docker**, que dá acesso ao daemon do Docker sem precisar de sudo toda vez.

docker ps vs. sudo docker ps



```
docker ps
```

permission denied

O daemon do Docker roda com privilégios de root. Um usuário comum, fora do grupo docker, não tem permissão para falar com ele diretamente.



```
sudo docker ps
```

funciona (lista os containers)

sudo eleva a permissão na hora (pedindo a senha). Já adicionar o usuário ao grupo docker (com logout/login) faz o docker ps funcionar sem sudo.

Recapitulando tudo

- ✓ Compartilhar hardware entre aplicações economiza dinheiro, energia e tempo
- ✓ A linha do tempo foi: 1 app por máquina → máquinas virtuais → containers (Docker)
- ✓ Docker compartilha o kernel do SO; a VM carrega um SO completo em cada instância
- ✓ Linux domina os servidores da internet — leve, estável, gratuito
- ✓ Servidores não têm tela: por isso `cd`, `ls`, `cat`, `nano`, `mkdir` e `rm` são essenciais
- ✓ SSH é a ponte segura até um servidor sem teclado, mouse ou monitor
- ✓ Docker: Dockerfile (receita) → imagem (molde) → container (em execução) → Docker Hub
- ✓ Grupos Linux (como o grupo `docker`) definem quem usa certos comandos sem `sudo`

Dúvidas?